Вычислительная математика

Домашнее задание № 1. Вариант 25

Тимофей Рыко

17 октября 2026 года

1. Что требуется сделать

Для двух функций нужно построить интерполяционный полином и кубический сплайн. В каждом случае берутся от 20 до 60 узлов: сначала равномерных, затем чебышёвских. Нужно сравнить точность методов, получить оценки ошибки и проверить влияние чётности степени полинома.

$$f_1(x) = \cos x, \quad x \in [0, 2\pi]; \qquad f_2(x) = \tanh x, \quad x \in [-\pi, \pi].$$

Интерполяция означает, что построенная функция точно совпадает с исходной во всех выбранных узлах. Между узлами они могут различаться. Эти различия и исследуются в работе.

1.1. Какие узлы используются

N — число узлов. Полином через N точек имеет степень не выше N-1. Например, при N=20 его степень не выше 19. Пусть [a,b] — заданный отрезок, L=b-a=2π.

Равномерная сетка: расстояния между соседними узлами одинаковы, крайние узлы совпадают с a и b.

$$x_j = a + \frac{j(b-a)}{N-1}, \qquad j = 0, \ldots, N-1.$$

Сетка Чебышёва: узлы сгущены к концам отрезка. По условию используются нули полинома Чебышёва T_N:

$$x_j = \frac{a+b}{2} + \frac{b-a}{2} \cos \frac{(2j+1)\pi}{2N}, \qquad j = 0, \ldots, N-1.$$

Эти узлы не включают сами концы отрезка. Перед расчётом они сортируются по возрастанию. В обоих случаях узлов ровно N.

1.2. Как сравнивается точность

В каждой точке берётся модуль разности исходной функции и приближения. Затем находится наибольшее значение на всём заданном отрезке:

$$E_P = \max_{[a,b]} |f(x) - P(x)|, \qquad E_S = \max_{[a,b]} |f(x) - S(x)|.$$

P обозначает полином, S — сплайн. Чем меньше E, тем точнее приближение. В таблицах максимум найден численно. Априорная оценка B — это выведенная из формул верхняя граница: ошибка должна удовлетворять $E \leq B$.

2. Интерполяционный полином

Через N различных узлов проходит единственный полином степени не выше N-1. Для расчёта выбрана форма Ньютона:

$$P(x) = c_0 + \sum_{k=1}^{N-1} c_k \prod_{j=0}^{k-1} (x - x_j).$$

Коэффициенты c_k — разделённые разности. Начальный столбец состоит из значений функции. Каждый следующий получается вычитанием соседних элементов и делением на расстояние между крайними узлами:

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Значение полинома считается вложенным умножением, как в схеме Горнера. Решать большую систему для коэффициентов при степенях x не требуется.

2.1. Оценка погрешности

Если у функции есть непрерывная производная порядка N, остаток интерполяции имеет вид:

$$f(x) - P(x) = \frac{f^{(N)}(\xi)}{N!} \prod_{j=0}^{N-1} (x - x_j), \qquad \xi \in [a, b].$$

Пусть M_N — верхняя граница модуля этой производной. Обозначим произведение расстояний до узлов через ω_N. Тогда:

$$E_P \le \frac{M_N}{N!} \max_{[a,b]} |\omega_N(x)|.$$

Равномерная сетка. При шаге h=L/(N-1) произведение расстояний до двух ближайших узлов не больше $h^2/4$. Остальные множители можно ограничить последовательными кратными h. Отсюда:

$$\max |\omega_N| \le \frac{h^N (N-1)!}{4}, \qquad B_P = \frac{M_N h^N}{4N}.$$

Нули Чебышёва. Здесь ω_N пропорционально T_N, а $|T_N| \le 1$ на [-1,1]. С учётом длины отрезка:

$$\max |\omega_N| = 2 \left(\frac{L}{4}\right)^N, \qquad B_P = \frac{2M_N}{N!} \left(\frac{L}{4}\right)^N.$$

На следующей странице показано, как получить M_N для обеих функций. Эти оценки относятся к самому полиному. Погрешность округления компьютера рассматривается отдельно.

3. Как оцениваются производные

3.1. Функция cos x

Производные cos x последовательно равны -sin x, -cos x, sin x, cos x и затем повторяются. Их модуль не больше 1. Поэтому для любого N можно взять $\mathbf{M_N=1}$ и подставить его в формулы предыдущей страницы.

3.2. Функция tanh x

Чтобы не выписывать вручную производные до 60-го порядка, используется простое рекуррентное правило. Положим t=tanh x. Тогда $t'=1-t^2$, поэтому каждая производная tanh x является полиномом от t:

$$Q_0(t) = t, \qquad Q_{k+1}(t) = (1 - t^2)Q_k'(t), \qquad f^{(k)}(x) = Q_k(\tanh x).$$

Например, первые два шага дают:

$$Q_1(t) = 1 - t^2, \qquad Q_2(t) = -2t + 2t^3.$$

Так как $|\tanh x|\le 1$, модуль каждого t^j также не больше 1. Следовательно, модуль полинома не превышает сумму модулей его коэффициентов:

$$Q_N(t) = \sum_j a_j t^j \quad \Longrightarrow \quad |f^{(N)}(x)| \le \sum_j |a_j| =: M_N.$$

Программа строит коэффициенты Q_N по указанному правилу. Они целые, поэтому эта часть расчёта выполняется без округления. Полученное M_N подставляется в оценки для обеих сеток.

Почему оценка может быть большой? При суммировании модулей теряется взаимное сокращение слагаемых. Поэтому такая граница для tanh x сильно завышена. Она подходит для проверки $E\le B$, но по ней нельзя судить, сходится ли интерполяция. Большая верхняя граница не означает большую фактическую ошибку.

3.3. Что означает чётность степени

В задании степень удобно обозначать n=N-1. Но это верхняя граница: некоторые старшие коэффициенты могут оказаться нулевыми. На симметричной сетке полином сохраняет симметрию функции. Поэтому полином для cos x чётен относительно центра π, а для tanh x нечётен относительно нуля.

При сравнении чётности используются соседние пары N=20 и 21, 22 и 23, …, 58 и 59. Это позволяет видеть влияние чётности отдельно от общего увеличения числа узлов. Результаты для разных сеток рассматриваются отдельно.

4. Кубический сплайн

Полином строится сразу на всём отрезке. Сплайн устроен иначе: между каждой парой соседних узлов строится свой кубический полином. В узлах соседние части соединяются так, чтобы совпадали значение, первая и вторая производные.

Используется **естественный кубический сплайн**, как в семинаре 6: на краях сетки его вторая производная равна нулю. Это выбранные граничные условия для сплайна Шёнберга.

$$S(x_i) = f(x_i), \qquad S \in C^2, \qquad S''(x_0) = S''(x_{N-1}) = 0.$$

Пусть $h_i=x_{i+1}-x_i$, $y_i=f(x_i)$, $m_i=S''(x_i)$. Неизвестные вторые производные находятся из системы для i=1,…,N-2:

$$h_{i-1}m_{i-1} + 2(h_{i-1}+h_i)m_i + h_i m_{i+1} = 6\left(\frac{y_{i+1}-y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right).$$

После этого кубический участок определяется однозначно. Если $u=(x-x_i)/h_i$, а L_i — прямая между соседними значениями функции, то:

$$S(x) = L_i(x) + \frac{h_i^2}{6}\left[m_i((1-u)^3 - (1-u)) + m_{i+1}(u^3 - u)\right].$$

На сетке Чебышёва крайние узлы не совпадают с концами отрезка. На оставшихся коротких участках сплайн продолжается касательными прямыми. Например, слева $S(x)=S(x_0)+S'(x_0)(x-x_0)$. Новые узлы не добавляются; значение и первые две производные остаются непрерывными.

4.1. Оценка погрешности сплайна

Обозначим $H=\max h_i$ и $M_2=\max|f''|$. Из системы для m_i получается $\max|m_i|\le 3M_2$: диагональный коэффициент вдвое больше суммы соседних, а правая часть ограничена через M_2. Ошибка прямой L_i не больше $M_2h_i^2/8$, отличие сплайна от этой прямой — не больше $3M_2h_i^2/8$. Складывая, получаем:

$$E_S \le \frac{M_2 H^2}{2} \qquad \text{между крайними узлами.}$$

Для линейного продолжения на длину δ слева добавляется граница $M_2(\delta h_0+\delta^2/2)$, справа — аналогичная. В программе B_S равна максимуму внутренней и двух крайних границ.

$$M_2(\cos) = 1, \qquad M_2(\tanh) = \frac{4}{3\sqrt{3}}.$$

Последнее значение получается из $f''=-2t(1-t^2)$, $t=\tanh x$: максимум модуля достигается при $|t|=1/\sqrt{3}$. Полученная оценка имеет порядок H^2. Обещать порядок H^4 на всём отрезке здесь нельзя: естественные условия $S''=0$ не совпадают с f'' на краях.

5. Как выполнен расчёт

1. Построение. Для каждого N от 20 до 60 строятся обе сетки. В узлах вычисляются cos x или tanh x. По этим значениям строятся полином Ньютона и естественный сплайн. Всего получается 164 набора данных и 328 приближений.

2. Поиск ошибки. Приближения сравниваются с исходной функцией на объединении двух проверочных сеток: 2001 равномерной точки и 2001 точки со сгущением к краям. Затем уточняется положение пяти наибольших локальных пиков ошибки. Концы исходного отрезка также проверяются.

3. Почему повышена точность. У cos x ошибка полинома становится намного меньше 10^{-16}. Обычная арифметика компьютера уже не позволяет её различить. Поэтому полином считается с 90 десятичными знаками. Каждый найденный максимум дополнительно проверяется другой формой того же полинома при 120 знаках.

4. Проверки. Подтверждено совпадение приближений с функцией в узлах, выполнение условий $S''=0$ и неравенств $E \leq B$. Для N=20,21,40,41,59,60 расчёт дополнительно проверен на сетке из 8001 точки. Существенно больших ошибок на ней не найдено. Табличный максимум остаётся численной оценкой, а не точным аналитическим значением.

5.1. Ошибка полинома и ошибка округления

cos x, N	Сетка	Ошибка P, 90 знаков	Ошибка расчёта float64
20	равномерная	1.077e-12	1.750e-12
20	Чебышёв	6.178e-15	6.772e-15
40	равномерная	3.799e-35	5.676e-07
40	Чебышёв	1.619e-40	7.772e-16
60	равномерная	5.357e-62	4.931e-01
60	Чебышёв	1.352e-70	9.992e-16

Float64 — обычные вычисления примерно с 16 значащими десятичными цифрами. В последнем столбце приведена ошибка библиотечного барицентрического интерполятора SciPy на проверочной сетке.

На равномерной сетке при большом N даже малые округления узлов и значений сильно влияют на результат. Поэтому при N=60 обычный расчёт cos x ошибается почти на 0,5, хотя сам интерполяционный полином очень точен. На сетке Чебышёва округление ограничивает результат уровнем около 10^{-15}. Основные таблицы ниже содержат высокоточные полиномиальные результаты.

Сплайн считается обычной арифметикой: его ошибки здесь достаточно велики, чтобы надёжно отличаться от округления.

6. Результаты для cos x

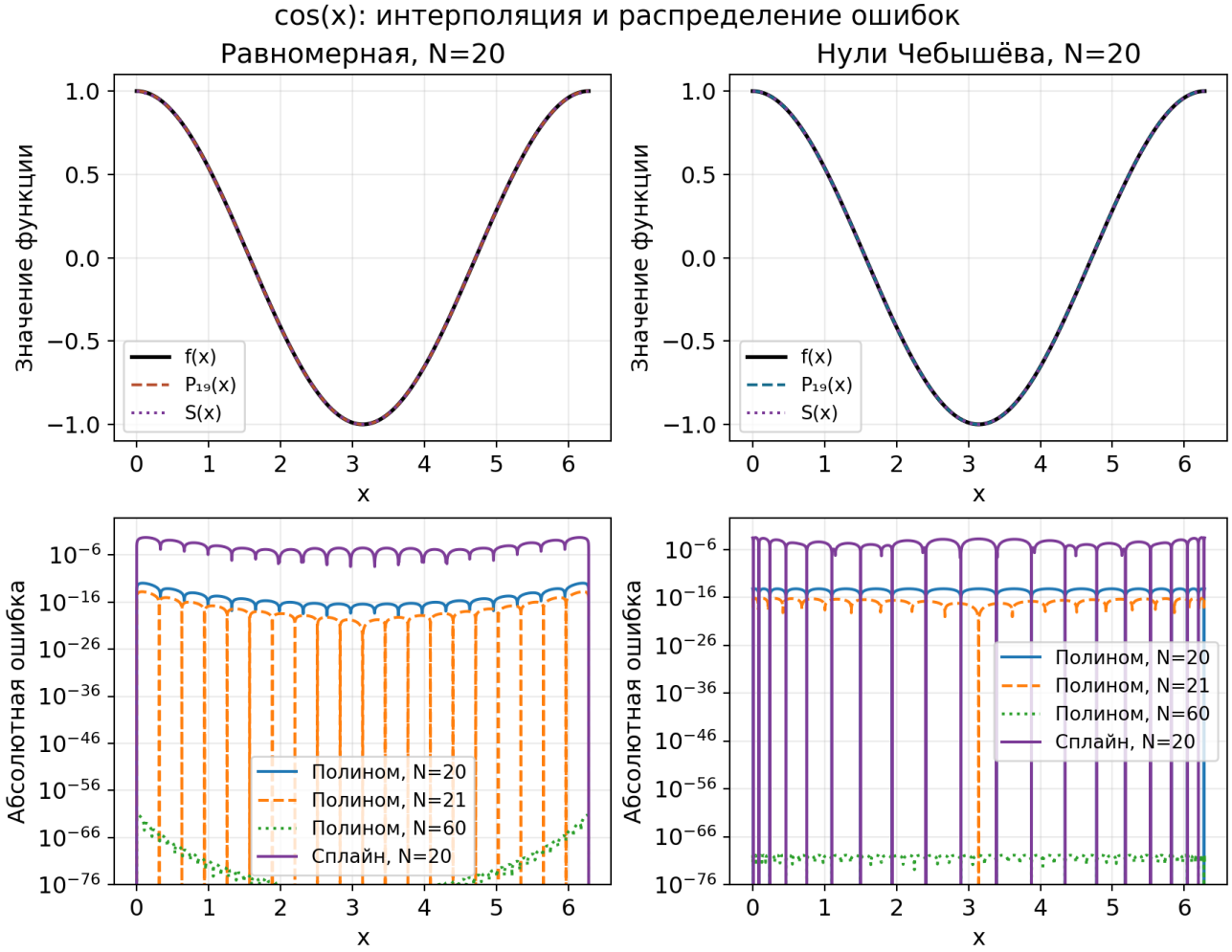


Рис. 1. Сверху — функция, полином и сплайн при N=20. Снизу — абсолютные ошибки. По вертикали в нижних графиках логарифмическая шкала: каждый следующий порядок означает изменение ошибки в 10 раз.

N	P: равномерная	P: Чебышёв	S: равномерная	S: Чебышёв
20	1.077e-12	6.178e-15	5.446e-03	3.204e-04
21	1.611e-14	6.566e-17	4.908e-03	2.638e-04
40	3.799e-35	1.619e-40	1.279e-03	2.020e-05
41	1.562e-37	4.634e-43	1.215e-03	1.830e-05
59	7.321e-62	2.699e-70	5.770e-04	4.275e-06
60	5.357e-62	1.352e-70	5.576e-04	3.997e-06

Полином быстро приближает cos x на обеих сетках. При N=60 чебышёвские узлы дают ошибку $1{,}352 \cdot 10^{-70}$, равномерные — $5{,}357 \cdot 10^{-62}$. На верхних графиках полином практически совпадает с функцией; разницу показывает график ошибки.

Сплайн менее точен. Его наибольшая ошибка на равномерной сетке возникает около концов: у cos x там $f'' = -1$, а у естественного сплайна $S'' = 0$. Чебышёвские узлы расположены плотнее у границ и уменьшают эту ошибку. При N=60 такой сплайн примерно в 140 раз точнее равномерного.

6.1. cos x: оценки и чётность

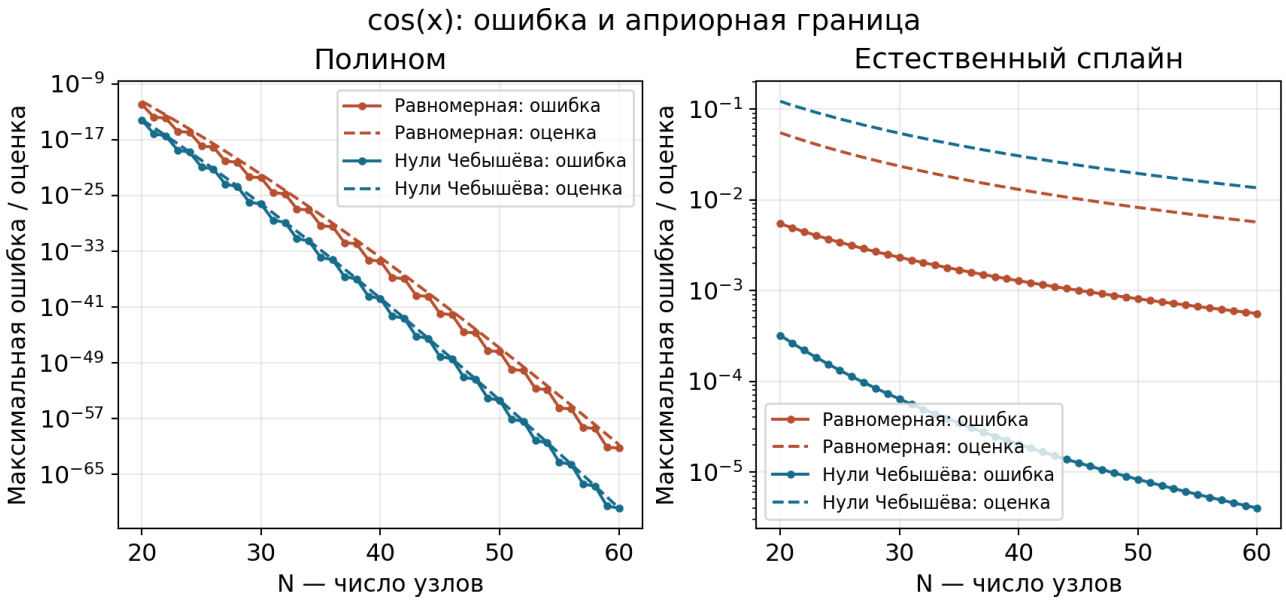


Рис. 2. Ошибка при всех N от 20 до 60 и её априорная граница. Сплошная линия — ошибка, пунктирная — граница. Чем ниже линия, тем меньше соответствующая величина.

N	Сетка	E_P	B_P	E_S	B_S
20	равн.	1.077e-12	3.058e-12	5.446e-03	5.468e-02
20	Чеб.	6.178e-15	6.875e-15	3.204e-04	1.215e-01
60	равн.	5.357e-62	1.817e-61	5.576e-04	5.671e-03
60	Чеб.	1.352e-70	1.406e-70	3.997e-06	1.353e-02

Все ошибки ниже соответствующих границ. Для полинома при N=60 на сетке Чебышёва оценка завышает ошибку всего примерно на 4%. Оценка сплайна заметно грубее, поскольку использует только максимальный шаг и не учитывает, где именно сгущены узлы.

Влияние чётности. Относительно центра π функция cos x чётна: cos(π+t)=-cos t. Интерполяционный полином тоже чётен по t. При N=20 разрешена степень до 19, но нечётный старший член равен нулю: фактическая степень не выше 18. При N=21 доступна степень 20. Поэтому переход 20→21 резко улучшает точность. Следующий переход 21→22 оставляет ту же доступную чётную степень 20.

Во всех соседних парах (20,21),…,(58,59) меньше ошибка у нечётного N на обеих сетках. То есть в обозначении n=N-1 предпочтительна **чётная степень**. Это сравнение соседних пар: оно не означает, что любое нечётное N лучше любого чётного.

7. Результаты для tanh x

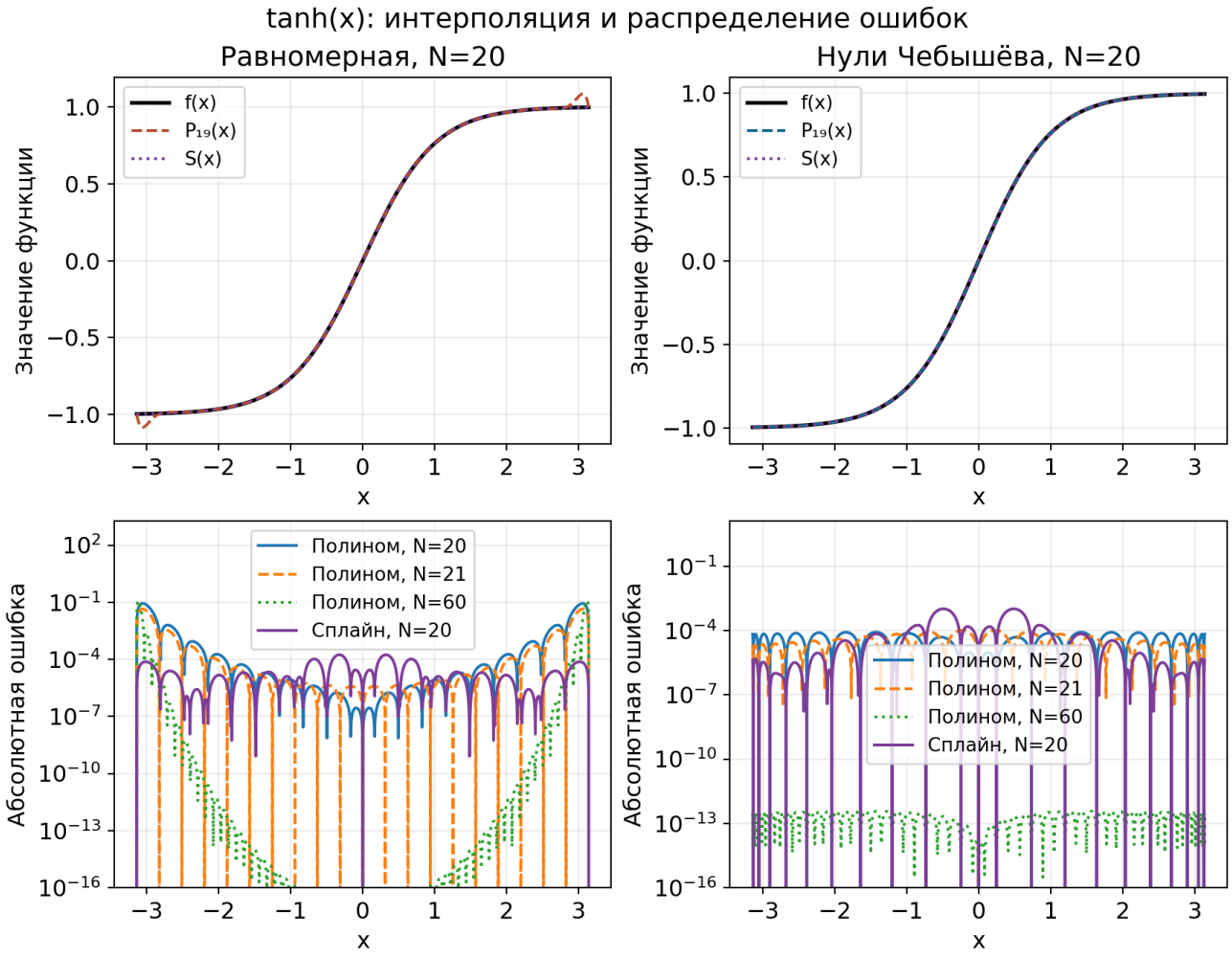


Рис. 3. Функция, приближения и абсолютные ошибки. У равномерного полинома заметны отклонения около концов отрезка. Чебышёвский полином приближает функцию всё точнее с ростом N.

N	P: равномерная	P: Чебышёв	S: равномерная	S: Чебышёв
20	8.726e-02	8.378e-05	1.744e-04	1.030e-03
21	4.428e-02	1.018e-04	1.304e-04	8.405e-04
40	9.020e-02	5.559e-09	1.863e-05	4.189e-05
41	4.604e-02	6.838e-09	1.772e-05	4.107e-05
59	5.839e-02	1.187e-12	8.490e-06	8.706e-06
60	1.179e-01	3.678e-13	8.206e-06	7.938e-06

На равномерной сетке увеличение числа узлов не решает проблему. Сначала ошибка немного снижается, затем растёт. Минимум среди чётных N достигается при N=28 (≈0,0841), среди нечётных — при N=27 (≈0,0428). При N=60 ошибка уже равна 0,1179. Такие краевые колебания полинома называют **эффектом Рунге**.

На сетке Чебышёва ошибка уменьшается до $3{,}678 \cdot 10^{-13}$. Сплайны на обеих сетках также становятся точнее и при N=60 дают близкие ошибки около $8 \cdot 10^{-6}$. При малом N равномерный сплайн лучше: в центральной области быстрого изменения tanh x у него больше узлов.

7.1. tanh x: оценки и чётность

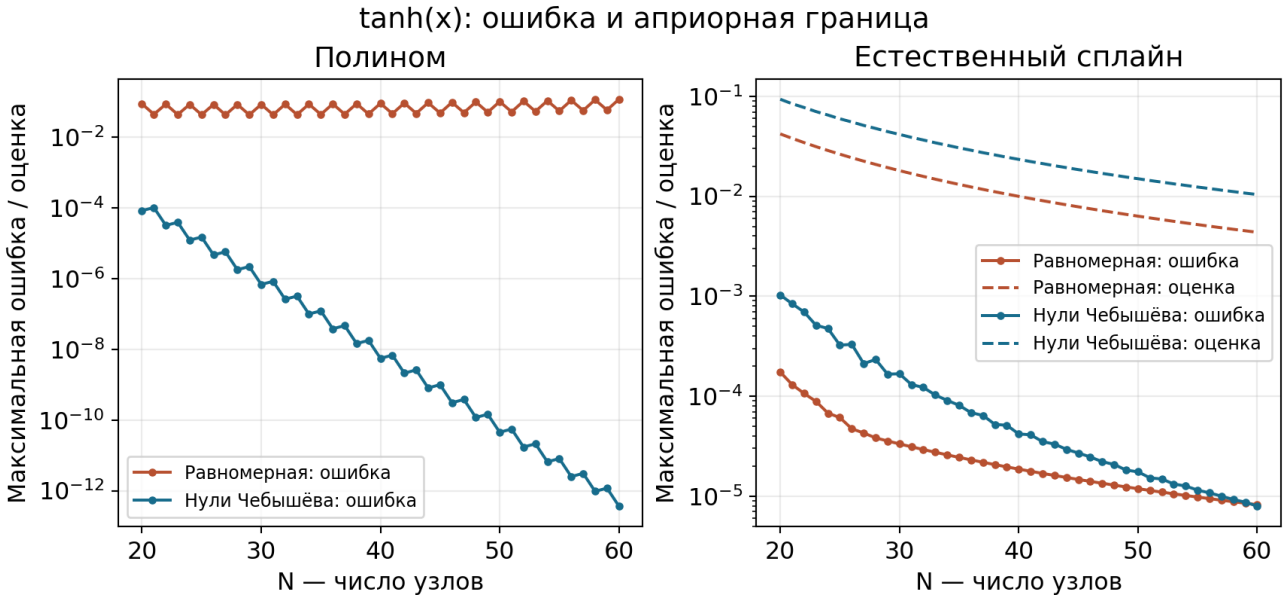


Рис. 4. Слева — ошибки полиномов, справа — ошибки сплайнов и их границы. Грубые оценки полиномиальной ошибки вынесены в таблицу: на общем графике их большие значения скрыли бы различия фактических ошибок.

N	Сетка	E_P	B_P	E_S	B_S
20	равн.	8.726e-02	1.188e+09	1.744e-04	4.209e-02
20	Чеб.	8.378e-05	2.670e+06	1.030e-03	9.354e-02
60	равн.	1.179e-01	3.794e+27	8.206e-06	4.365e-03
60	Чеб.	3.678e-13	2.936e+18	7.938e-06	1.041e-02

Неравенства $E \le B$ выполнены, но полиномиальная оценка сильно завышена. Причина — грубая граница для высокой производной: сумма модулей коэффициентов не учитывает их сокращение. Поэтому эта оценка не объясняет хорошее поведение чебышёвского полинома. Оно установлено численным расчётом.

Влияние чётности. Функция tanh x нечётна, и её интерполяционный полином тоже нечётен. При N=20 и N=21 фактическая степень не выше 19 в обоих случаях. При этом узлы различаются, поэтому совпадения ошибок ожидать нельзя.

Сетка Чебышёва: во всех 20 парах (20,21),…,(58,59) лучше чётное N. Следовательно, в обозначении n=N-1 лучше **нечётная степень**. Например, при N=20 ошибка $8{,}378 \cdot 10^{-5}$, при N=21 — $1{,}018 \cdot 10^{-4}$.

Равномерная сетка: во всех тех же парах лучше нечётное N, то есть **чётная степень n=N-1**. Фактический полином при этом всё равно нечётный: коэффициент при старшей чётной степени равен нулю. Например, ошибки при N=20 и 21 равны 0,08726 и 0,04428. Значит, единого правила чётности для всех сеток здесь нет.

8. Выводы

1. Для cos x полиномиальная интерполяция точнее сплайна. Ошибка быстро уменьшается на обеих сетках, особенно на чебышёвской. Но для больших N нужны вычисления с повышенной точностью, иначе результат искажает округление.

2. Для tanh x важно выбрать правильную сетку. Равномерный полином имеет краевые колебания, которые не исчезают при увеличении N до 60. Чебышёвская сетка даёт быстрое уменьшение ошибки: при N=60 она около $3{,}68 \cdot 10^{-13}$.

3. Сплайн даёт устойчивый результат для обеих функций. Он уступает хорошему полиному по точности, но не испытывает сильных краевых колебаний высокой степени. Его ошибка зависит от граничных условий. Для cos x сгущение узлов у концов заметно улучшает результат.

4. Априорная оценка — это граница, а не прогноз точного значения. Для cos x остаточная оценка полинома достаточно близка к ошибке. Для tanh x применённая простая оценка производной сильно завышает её. Все проверенные ошибки находятся ниже выведенных границ.

5. Чётность зависит от функции и сетки. В соседних парах для cos x предпочтительна чётная степень n=N-1 на обеих сетках. Для tanh x на чебышёвской сетке лучше нечётная степень, на равномерной — чётная. Нужно различать степень n=N-1 и фактическую степень после обнуления коэффициентов.

9. Источники и программа

1. «Задание 1.pdf», с. 10 — условие варианта 25.

2. «Семинар 6 осень_ Сплайны.pdf», с. 1–3 — кубический сплайн и естественные граничные условия.

3. SciPy: CubicSpline — построение сплайна; BarycentricInterpolator — сравнение с обычной арифметикой.

4. mpmath и Python Decimal — повышенная точность; NumPy — массивы; Matplotlib — графики.

Версии: python 3.12.10, numpy 2.4.6, scipy 1.17.1, mpmath 1.3.0, matplotlib 3.10.9.

Код приведён полностью в приложении Б и сохранён отдельным файлом «Рыко_ДЗ1_вариант25.ру». Запуск: `python -X utf8 Рыко_ДЗ1_вариант25.ру`. Скрипт создаёт таблицу «ошибки.csv» и графики в папке «результаты».

В приложении А приведены ошибки для всех N от 20 до 60. Запись 1.234e-05 означает $1{,}234 \cdot 10^{-5}$.

Приложение A. Полные таблицы ошибок

Функция $\cos x$

P — полином, S — естественный сплайн. N — число узлов. Запись 1.234e-05 означает $1{,}234 \cdot 10^{-5}$.

N	P: равномерная	P: Чебышёв	S: равномерная	S: Чебышёв
20	1.077e-12	6.178e-15	5.446e-03	3.204e-04
21	1.611e-14	6.566e-17	4.908e-03	2.638e-04
22	1.168e-14	3.328e-17	4.446e-03	2.193e-04
23	1.468e-16	2.962e-19	4.047e-03	1.837e-04
24	1.066e-16	1.498e-19	3.700e-03	1.551e-04
25	1.142e-18	1.133e-21	3.395e-03	1.318e-04
26	8.297e-19	5.723e-22	3.127e-03	1.127e-04
27	7.666e-21	3.724e-24	2.889e-03	9.696e-05
28	5.575e-21	1.878e-24	2.677e-03	8.388e-05
29	4.488e-23	1.062e-26	2.488e-03	7.292e-05
30	3.266e-23	5.352e-27	2.319e-03	6.370e-05
31	2.311e-25	2.656e-29	2.166e-03	5.589e-05
32	1.683e-25	1.337e-29	2.028e-03	4.924e-05
33	1.055e-27	5.868e-32	1.902e-03	4.355e-05
34	7.691e-28	2.951e-32	1.788e-03	3.865e-05
35	4.302e-30	1.154e-34	1.684e-03	3.443e-05
36	3.137e-30	5.799e-35	1.589e-03	3.077e-05
37	1.575e-32	2.032e-37	1.501e-03	2.758e-05
38	1.149e-32	1.021e-37	1.421e-03	2.479e-05
39	5.206e-35	3.224e-40	1.347e-03	2.235e-05
40	3.799e-35	1.619e-40	1.279e-03	2.020e-05
41	1.562e-37	4.634e-43	1.215e-03	1.830e-05
42	1.140e-37	2.326e-43	1.156e-03	1.662e-05
43	4.271e-40	6.059e-46	1.102e-03	1.513e-05
44	3.119e-40	3.040e-46	1.051e-03	1.380e-05
45	1.069e-42	7.240e-49	1.004e-03	1.262e-05
46	7.810e-43	3.632e-49	9.595e-04	1.156e-05
47	2.459e-45	7.936e-52	9.182e-04	1.061e-05
48	1.797e-45	3.980e-52	8.794e-04	9.751e-06
49	5.214e-48	8.009e-55	8.431e-04	8.980e-06
50	3.811e-48	4.015e-55	8.089e-04	8.283e-06
51	1.022e-50	7.465e-58	7.768e-04	7.653e-06
52	7.475e-51	3.742e-58	7.466e-04	7.081e-06
53	1.860e-53	6.447e-61	7.181e-04	6.562e-06
54	1.360e-53	3.231e-61	6.912e-04	6.090e-06
55	3.147e-56	5.173e-64	6.658e-04	5.659e-06
56	2.302e-56	2.592e-64	6.418e-04	5.266e-06
57	4.965e-59	3.867e-67	6.190e-04	4.906e-06
58	3.632e-59	1.938e-67	5.975e-04	4.577e-06
59	7.321e-62	2.699e-70	5.770e-04	4.275e-06
60	5.357e-62	1.352e-70	5.576e-04	3.997e-06

Функция $\tanh x$

P — полином, S — естественный сплайн. N — число узлов. Запись 1.234e-05 означает $1{,}234 \cdot 10^{-5}$.

N	P: равномерная	P: Чебышёв	S: равномерная	S: Чебышёв
20	8.726e-02	8.378e-05	1.744e-04	1.030e-03
21	4.428e-02	1.018e-04	1.304e-04	8.405e-04
22	8.562e-02	3.205e-05	1.065e-04	6.958e-04
23	4.349e-02	3.902e-05	8.863e-05	5.122e-04
24	8.462e-02	1.228e-05	6.767e-05	4.757e-04
25	4.303e-02	1.495e-05	6.108e-05	3.237e-04
26	8.415e-02	4.678e-06	4.719e-05	3.303e-04
27	4.282e-02	5.721e-06	4.282e-05	2.116e-04
28	8.411e-02	1.788e-06	3.824e-05	2.332e-04
29	4.282e-02	2.189e-06	3.564e-05	1.661e-04
30	8.443e-02	6.844e-07	3.329e-05	1.675e-04
31	4.301e-02	8.374e-07	3.116e-05	1.307e-04
32	8.506e-02	2.609e-07	2.923e-05	1.223e-04
33	4.335e-02	3.202e-07	2.747e-05	1.028e-04
34	8.598e-02	9.971e-08	2.587e-05	9.075e-05
35	4.384e-02	1.224e-07	2.440e-05	8.103e-05
36	8.715e-02	3.814e-08	2.305e-05	6.835e-05
37	4.446e-02	4.681e-08	2.181e-05	6.421e-05
38	8.856e-02	1.455e-08	2.066e-05	5.220e-05
39	4.519e-02	1.789e-08	1.961e-05	5.119e-05
40	9.020e-02	5.559e-09	1.863e-05	4.189e-05
41	4.604e-02	6.838e-09	1.772e-05	4.107e-05
42	9.204e-02	2.125e-09	1.688e-05	3.506e-05
43	4.700e-02	2.613e-09	1.609e-05	3.318e-05
44	9.410e-02	8.111e-10	1.536e-05	2.937e-05
45	4.806e-02	9.986e-10	1.468e-05	2.699e-05
46	9.636e-02	3.099e-10	1.404e-05	2.465e-05
47	4.923e-02	3.816e-10	1.345e-05	2.210e-05
48	9.882e-02	1.184e-10	1.288e-05	2.075e-05
49	5.050e-02	1.458e-10	1.236e-05	1.821e-05
50	1.015e-01	4.521e-11	1.186e-05	1.752e-05
51	5.187e-02	5.571e-11	1.140e-05	1.523e-05
52	1.043e-01	1.727e-11	1.096e-05	1.485e-05
53	5.334e-02	2.129e-11	1.055e-05	1.321e-05
54	1.074e-01	6.601e-12	1.015e-05	1.262e-05
55	5.492e-02	8.132e-12	9.784e-06	1.148e-05
56	1.107e-01	2.520e-12	9.434e-06	1.078e-05
57	5.660e-02	3.107e-12	9.103e-06	9.987e-06
58	1.142e-01	9.626e-13	8.788e-06	9.232e-06
59	5.839e-02	1.187e-12	8.490e-06	8.706e-06
60	1.179e-01	3.678e-13	8.206e-06	7.938e-06

Приложение Б. Полный код программы

```
"""ДЗ1, вариант 25.

Источники: «Задание 1.pdf», с. 10; «Семинар 6 осень_ Сплайны.pdf», с. 1–3;
https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html
https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.BarycentricInterpolator.html
https://mpmath.org/doc/current/ ; https://docs.python.org/3/library/decimal.html
Запуск: python -X utf8 Рыко_ДЗ1_вариант25.py
Зависимости: numpy scipy matplotlib mpmath. Время: несколько минут.
N — число узлов, номинальная степень полинома — N-1.
"""
from pathlib import Path
from decimal import Decimal, getcontext
import csv
import json
import platform
import numpy as np
import scipy
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline, BarycentricInterpolator
from scipy.optimize import minimize_scalar
import mpmath as mp

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "результаты"
OUT.mkdir(exist_ok=True)
getcontext().prec = 90
mp.mp.dps = 110
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 12})


def dec(x):
    # 105 значащих цифр перед округлением арифметики Decimal до 90.
    return Decimal(mp.nstr(x, 105))


def nodes(n, grid, a, b):
    if grid == "uniform":
        return [a + (b-a)*j/(n-1) for j in range(n)]
    return sorted([(a+b)/2 + (b-a)/2*mp.cos((2*j+1)*mp.pi/(2*n))
                   for j in range(n)])


def newton_coefficients(x, y):
    # Обратный порядок обновления сохраняет предыдущий столбец разностей.
    c = list(y)
    for k in range(1, len(x)):
        for j in range(len(x)-1, k-1, -1):
            c[j] = (c[j]-c[j-1])/(x[j]-x[j-k])
    return c


def polynomial(t, x, c):
```

```
    # Схема Горнера для формы Ньютона, без системы Вандермонда.
    value = c[-1]
    for j in range(len(c)-2, -1, -1):
        value = c[j] + (t-x[j])*value
    return value


def numerical_maximum(x, errors, error_at):
    # Сетка даёт кандидатов; уточняем пять наибольших локальных максимумов.
    candidates = np.where((errors[1:-1] >= errors[:-2]) &
                          (errors[1:-1] >= errors[2:]))[0] + 1
    candidates = sorted(candidates, key=lambda j: errors[j], reverse=True)[:5]
    best = (float(np.max(errors)), float(x[np.argmax(errors)]))
    for j in candidates:
        r = minimize_scalar(lambda t: -error_at(t),
                            bounds=(x[j-1], x[j+1]), method="bounded",
                            options={"xatol": 1e-14})
        best = max(best, (error_at(r.x), float(r.x)))
    return best  # Численная оценка, а не доказанный супремум.


def natural_extension(spline, x, y, t):
    # Вне крайних узлов — касательные прямые. Новых узлов не добавляем.
    t = np.asarray(t)
    v = spline(np.clip(t, x[0], x[-1]))
    v = np.where(t < x[0], y[0] + spline(x[0], 1)*(t-x[0]), v)
    return np.where(t > x[-1], y[-1] + spline(x[-1], 1)*(t-x[-1]), v)


def bounds(n, grid, x, a, b, name):
    length = b-a
    # Для cos модуль любой производной <= 1.
    # Для tanh положим t=tanh(x). Производные — полиномы Q_N(t):
    # Q_0=t, Q_(k+1)=(1-t*t)*Q_k'. При |t|<=1 их модуль
    # не больше суммы модулей коэффициентов. Все коэффициенты — целые.
    m_n = 1
    if name == "tanh":
        q = [0, 1]  # Коэффициенты при степенях t: от младшей к старшей.
        for _ in range(n):
            derivative = [j*q[j] for j in range(1, len(q))]
            q = derivative + [0, 0]
            for j, value in enumerate(derivative):
                q[j+2] -= value
        m_n = sum(abs(value) for value in q)
    derivative_factor = mp.mpf(m_n)/mp.factorial(n)
    omega = (length/(n-1))**n*mp.factorial(n-1)/4
    if grid == "chebyshev":
        omega = 2*(length/4)**n
    p_bound = derivative_factor*omega
    h = max(x[j+1]-x[j] for j in range(n-1))
    m2 = 1 if name == "cos" else 4/(3*mp.sqrt(3))
    left, right = x[0]-a, b-x[-1]
    s_bound = m2*max(h*h/2, left*(x[1]-x[0])+left*left/2,
                     right*(x[-1]-x[-2])+right*right/2)
    return float(p_bound), float(s_bound)


def independent_check(n, grid, a, b, f, peak, expected):
```

```
    # Независимая форма Лагранжа при 120 цифрах проверяет максимум Ньютона.
    with mp.workdps(120):
        aa, bb = (mp.mpf(0), 2*mp.pi) if a == 0 else (-mp.pi, mp.pi)
        x = nodes(n, grid, aa, bb)
        t = mp.mpf(str(peak))
        w = [1/mp.fprod(x[j]-x[k] for k in range(n) if j != k)
             for j in range(n)]
        p = mp.fsum(wj*f(xj)/(t-xj) for wj, xj in zip(w, x))
        p /= mp.fsum(wj/(t-xj) for wj, xj in zip(w, x))
        measured = float(abs(p-f(t)))
    relative = abs(measured-expected)/expected
    assert relative < 1e-9, (n, grid, relative)
    return relative


def main():
    rows, curves, checks = [], {}, []
    for name, f, ff, a, b in [("cos", mp.cos, np.cos, mp.mpf(0), 2*mp.pi),
                              ("tanh", mp.tanh, np.tanh, -mp.pi, mp.pi)]:
        # Объединяем равномерную и сгущённую к краям проверочные сетки.
        mesh = sorted(set([a+(b-a)*j/2000 for j in range(2001)] +
                          [(a+b)/2+(b-a)/2*mp.cos(mp.pi*j/2000)
                           for j in range(2001)]))
        xd, yd = [dec(t) for t in mesh], [dec(f(t)) for t in mesh]
        xf = np.array([float(t) for t in mesh])
        yf = np.array([float(f(t)) for t in mesh])
        for grid in ["uniform", "chebyshev"]:
            for n in range(20, 61):
                xm = nodes(n, grid, a, b)
                xx, yy = [dec(t) for t in xm], [dec(f(t)) for t in xm]
                c = newton_coefficients(xx, yy)
                assert max(abs(polynomial(t, xx, c)-v) for t, v in zip(xx, yy)) <
                ↪  Decimal('1e-65')
                pd = [polynomial(t, xx, c) for t in xd]
                pe = np.array([float(abs(v-y)) for v, y in zip(pd, yd)])

                def ep(t):
                    q = mp.mpf(str(t))
                    return float(abs(polynomial(dec(q), xx, c)-dec(f(q))))

                pmax, pwhere = numerical_maximum(xf, pe, ep)
                delta = independent_check(n, grid, a, b, f, pwhere, pmax)
                x = np.array([float(t) for t in xm])
                y = np.array([float(f(t)) for t in xm])
                s = CubicSpline(x, y, bc_type="natural")
                assert np.max(abs(s(x)-y)) < 1e-12
                assert np.max(abs(s(x[[0, -1]], 2))) < 1e-10
                sv = natural_extension(s, x, y, xf)
                se = abs(sv-yf)
                es = lambda t: float(abs(natural_extension(s, x, y, t)-ff(t)))
                smax, swhere = numerical_maximum(xf, se, es)
                bp, bs = bounds(n, grid, xm, a, b, name)
                assert pmax <= bp*(1+1e-9) and smax <= bs*(1+1e-9)
                # Обычная float64-реализация нужна только для сравнения округлений.
                p64 = BarycentricInterpolator(x, y, rng=0)(xf)
                row = dict(function=name, grid=grid, N=n, degree=n-1,
                           polynomial_error=pmax, spline_error=smax,
                           polynomial_bound=bp, spline_bound=bs,
```

```
                            float64_error=float(np.max(abs(p64-yf))),
                            x_polynomial_max=pwhere, x_spline_max=swhere,
                            independent_relative_difference=delta)
                rows.append(row)
                if n in [20, 21, 40, 41, 59, 60]:
                    # Вторая, более густая сетка проверяет поиск пиков.
                    dense = np.linspace(float(a), float(b), 8001)
                    dense_p = max(ep(t) for t in dense)
                    dense_s = float(np.max(abs(natural_extension(s, x, y,
                    ↪  dense)-ff(dense))))
                    assert dense_p <= pmax*1.00001 and dense_s <= smax*1.00001
                    checks.append(dict(function=name, grid=grid, N=n,
                                       dense_polynomial_ratio=dense_p/pmax,
                                       dense_spline_ratio=dense_s/smax))
                if n in [20, 21, 60]:
                    curves[f"{name}_{grid}_{n}"] = dict(x=xf, f=yf,
                        p=np.array([float(v) for v in pd]), s=sv, pe=pe, se=se)
                print(name, grid, n, f"P={pmax:.5e} S={smax:.5e}", flush=True)
    with (OUT / "ошибки.csv").open("w", encoding="utf-8-sig", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    (OUT / "проверки.json").write_text(json.dumps(dict(
        versions=dict(python=platform.python_version(), numpy=np.__version__,
                      scipy=scipy.__version__, mpmath=mp.__version__,
                      ↪  matplotlib=matplotlib.__version__),
        decimal_digits=90, rows=len(rows), dense_checks=checks,
        largest_independent_relative_difference=max(r['independent_relative_difference
        ↪  '] for r in rows)
    ), indent=2, ensure_ascii=False), encoding="utf-8")
    np.savez_compressed(OUT / "кривые.npz", **{f"{k}__{v}": arr
        for k, data in curves.items() for v, arr in data.items()})
    draw(rows, curves)


def draw(rows, curves):
    colors = {"uniform": "#B75030", "chebyshev": "#176C8C"}
    labels = {"uniform": "Равномерная", "chebyshev": "Нули Чебышёва"}
    for name in ["cos", "tanh"]:
        fig, axes = plt.subplots(2, 2, figsize=(9, 7), layout="constrained")
        for j, grid in enumerate(colors):
            v = curves[f"{name}_{grid}_20"]
            axes[0,j].plot(v['x'],v['f'],color='black',lw=2,label='f(x)')
            axes[0,j].plot(v['x'],v['p'],'--',color=colors[grid],label='P₁₉(x)')
            axes[0,j].plot(v['x'],v['s'],':',color='#7D3C98',label='S(x)')
            axes[0,j].set_title(labels[grid]+", N=20")
            for n, style in [(20,'-'),(21,'--'),(60,':')]:
                u=curves[f"{name}_{grid}_{n}"]
                axes[1,j].semilogy(u['x'],np.maximum(u['pe'],1e-85),style,
                                   label=f'Полином, N={n}')
            axes[1,j].semilogy(v['x'],np.maximum(v['se'],1e-85),color='#7D3C98',label=
            ↪  'Сплайн, N=20')
            axes[1,j].set_ylim(bottom=1e-76 if name=='cos' else 1e-16)
        for ax in axes.flat:
            ax.set_xlabel('x'); ax.grid(alpha=.25); ax.legend(fontsize=10)
        for ax in axes[0]: ax.set_ylabel('Значение функции')
        for ax in axes[1]: ax.set_ylabel('Абсолютная ошибка')
```

```
        fig.suptitle(f'{name}(x): интерполяция и распределение ошибок')
        fig.savefig(OUT / f'{name}_curves.png',dpi=190); plt.close(fig)
        fig, axes=plt.subplots(1,2,figsize=(9,4.2),layout='constrained')
        for grid in colors:
            rr=[r for r in rows if r['function']==name and r['grid']==grid]
            for ax,method in zip(axes,['polynomial','spline']):
                ns=[r['N'] for r in rr]
                ax.semilogy(ns,[r[method+'_error'] for r in rr],'-o',ms=3,
                            color=colors[grid],label=labels[grid]+': ошибка')
                # Грубая растущая граница для полинома tanh
                # дана в CSV и отчёте; здесь она скрыла бы полезные кривые.
                if not (name=='tanh' and method=='polynomial'):
                    ax.semilogy(ns,[r[method+'_bound'] for r in rr],'--',
                                color=colors[grid],label=labels[grid]+': оценка')
        for ax,title in zip(axes,['Полином','Естественный сплайн']):
            ax.set_title(title); ax.set_xlabel('N — число узлов')
            ax.set_ylabel('Максимальная ошибка / оценка')
            ax.grid(alpha=.25); ax.legend(fontsize=9)
        fig.suptitle(f'{name}(x): ошибка и априорная граница')
        fig.savefig(OUT/f'{name}_convergence.png',dpi=190); plt.close(fig)


if __name__ == "__main__":
    main()
```